

SQE ASSIGNMENT 3 - FINAL REPORT



OCTOBER 25, 2026

24F3122-Junaid Haider
24F3056-AbdulRehman

Table of Contents

1. Introduction 2
1.1 Purpose 2
1.2 Scope 2
1.3 Project under Test 2
1.4 Summary of Results 2
2. Task 1: Project Selection and Environment Setup 3
2.1 Environment 3
2.2 Setup Procedure 3
2.3 Setup Demonstration Video 3
3. Team and Module Allocation 3
4. Task 2: Test Design and Automated Unit Tests 4
4.1 Techniques Applied 4
4.2 Key Boundary Conditions 4
4.3 Test Coverage by Module and Function 4
5. Task 3: Data-Driven Testing and Excel Workbook 5
5.1 External CSV Data 5
5.2 Excel Test Case Workbook 6
6. Task 4: Suite Configuration, Ordering and Parallel Execution 6
6.1 Test Groups (Markers) 6
6.2 Parameterization 6
6.3 Execution Order 6
6.4 Parallel Execution 7
7. Execution and Verification Results 7
8. Deliverables and Traceability 7
9. Conclusion 8

1. Introduction

1.1 Purpose

This report documents the unit test automation solution built for SQE Assignment 3. It describes the selected open-source project, how the local environment was prepared, how the tests were designed, the data-driven and suite configuration, and the verified execution results.

1.2 Scope

The assignment brief defines four tasks. This report addresses each one and maps it to concrete evidence in the repository.

Table 1: *Assignment tasks addressed in this report*

Task	Requirement (summary)	Section
Task 1	Select an open-source Python project, set up the local environment, record a setup video	2
Task 2	Test three or more modules using ECP, BVA and similar techniques; 5–10 automated tests per function and per module	4
Task 3	At least 4 test cases written in an Excel sheet, consuming test data from an external CSV	5
Task 4	Configuration for parallel runs and suites, groups, parameterization and execution order	6

1.3 Project under Test

The selected project is **Expense Tracker**, a Python/Flask web application for recording and analysing personal expenses. Its pure-Python modules (validation, statistics, formatting, export helpers) are well suited to isolated unit testing. The original project ships with 113 regression tests, which were kept unchanged. The application source code in `expense_tracker/` was not modified.

1.4 Summary of Results

Table 2: *Headline numbers (verified by re-running the final project)*

Metric	Value
Original regression tests (unchanged)	113
New automated unit tests written by the group	128 (Abdul-Rehman 62, Junaid Haider 66)
Total suite	241 tests, all passing (241/241)
Modules tested	6 (3 per member)
External CSV data-driven cases	16 (8 per member)
Custom pytest markers / suites	10 module and technique suites

2. Task 1: Project Selection and Environment Setup

2.1 Environment

Table 3: *Verified development environment*

Component	Version / detail
Operating system	Windows (development); also verified on Linux
Python	3.14.3 (project supports 3.10 and above)
Flask	3.1.2
segno	1.6.6
pytest	8.4.1 (pluggy 1.7.0)
pytest-xdist	3.8.0 (parallel execution)
openpyxl	3.1.5 (Excel generation)
Virtual environment	`.venv` in the project root

2.2 Setup Procedure

1. Clone the group repository and open the project folder.
2. Create a virtual environment: `python -m venv .venv`.
3. Activate it: `.venv\Scripts\Activate.ps1` (Windows) or `source .venv/bin/activate` (Linux/macOS).
4. Install dependencies: `pip install -r requirements-dev.txt`.
5. Launch the application with `python app.py` and open `http://127.0.0.1:5000`.
6. Run the complete suite with `pytest` and confirm 241 tests pass.

2.3 Setup Demonstration Video

The setup video covers environment activation, dependency verification, launching the application, and running the serial, parallel and per-person test commands. The recording is kept in `sqe/video/`.

3. Team and Module Allocation

The brief requires three modules. To give each member independent, non-overlapping deliverables without editing the same files, coverage was extended to six modules, three per member. The split was balanced to 62 and 66 tests.

Table 4: *Ownership of modules, test files and design notes*

Member	Modules	Test files	Tests

Abdul-Rehman
24F-3056
	`models`
	`stats`
	`categories`
		`test_24F3056_models.py`
		`test_24F3056_stats.py`
		`test_24F3056_categories.py`
			62

Junaid Haider
24F-3122
	`csvutil`
	`options`
	`formatting`
		`test_24F3122_csvutil.py`
		`test_24F3122_options.py`
		`test_24F3122_formatting.py`
			66

Total	**6 modules**	**6 test files**	**128**

4. Task 2: Test Design and Automated Unit Tests

4.1 Techniques Applied

- **Equivalence Class Partitioning (ECP):** inputs are grouped into valid and invalid classes (for example valid categories versus unknown, empty or non-string values) and one representative of each class is tested.
- **Boundary Value Analysis (BVA):** tests sit on and just beyond each limit, such as description length 120 versus 121, amount $0.01 versus $0.00, and relative-time thresholds at 44 versus 45 seconds.
- **Security testing (CWE-1236):** CSV formula injection defence is tested for every dangerous prefix, while legitimate signed numbers must stay unchanged.
- **Contract and integration checks:** module-level tests verify constants, dataclasses and end-to-end pipelines.

Full reasoning for each module is documented in `sqe/docs/design_notes_<module>.md`.

4.2 Key Boundary Conditions

Table 5: *Selected boundary value analysis cases*

Area	Valid / lower side	Invalid / upper side
Description length	120 characters accepted	121 characters rejected
Amount (lower)	$0.01 accepted	$0.00 and negative rejected
Amount (upper)	$1,000,000.00 accepted	$1,000,000.01 rejected
Relative time	44 s gives “just now”; 3599 s gives minutes	45 s gives “1 min ago”; 3600 s gives “1 hr ago”
Calendar	1st and last day of a month; Feb 29, 2024	Non-ISO or non-existent dates raise errors
CSV prefixes	`+45.50`, `-120.00`, +8% kept as numbers	=, +cmd, @, tab, CR neutralised with a leading quote

4.3 Test Coverage by Module and Function

Each function has between 5 and 14 tests, and each module has a block of module-level tests (constants, dataclasses, pipelines). Counts marked with * include the 8 external-CSV cases from Task 3.

Table 6: *Automated unit tests per function and module-level block*

Member	Module	Function / scope	Technique focus	Tests
Abdul-Rehman	models	`validate_expense`	ECP, BVA	14*
		Dataclasses, constants (`Expense`, `Filters`, `CATEGORIES`)	Contract checks	6
Abdul-Rehman	stats	`total_of`	BVA, empty input	6
		`expenses_in_month`	BVA, month edges	6
		`category_breakdown`	BVA, ordering, percentages	6
		`monthly_trend`	BVA, year wrap, clamping	6
		Constants and dataclasses (donut geometry)	Contract checks	6
Abdul-Rehman	categories	`category_color`	ECP, fallback handling	6

Member	Module	Function / scope	Technique focus	Tests
		Palette constants and key parity	Contract checks	6
		Abdul-Rehman subtotal		**62**
Junaid Haider	csvutil	`neutralize`	Security, ECP, BVA	11*
		`csv_bytes`	Standards compliance, encoding	6
		Pipeline and constants	Integration	5
Junaid Haider	options	`sanitize_filename`	Security, BVA (80-char limit)	6
		`parse_export_options`	ECP, validation	6
		`ExportOptions` dataclass and stems	Contract checks	5
Junaid Haider	formatting	`format_currency`	ECP, extremes	6
		`format_date`	ECP, leap year, invalid input	5
		`format_relative`	BVA (time thresholds)	6
		`format_local`	ECP, noon/midnight, timezones	5
		Formatting package integration	Integration	5
		Junaid Haider subtotal		**66**
		Total new unit tests		**128**

5. Task 3: Data-Driven Testing and Excel Workbook

5.1 External CSV Data

Each member has one test that reads its cases from an external CSV file using `csv.DictReader` and runs them through `@pytest.mark.parametrize`. Changing the data requires no code change.

Table 7: *External data files and the tests that consume them*

Member	CSV file	Columns	Rows	Consumed by
Abdul-Rehman	`sqe/tests/data/test_24F3056_expenses_data.csv`	test_id, description, amount, category, date, expected_valid, error_field, error_message	8	`validate_expense` in `test_24F3056_models.py`
Junaid Haider	`sqe/tests/data/test_24F3122_data.csv`	test_id, input, expected, description	8	`neutralize` in `test_24F3122_csvutil.py`

5.2 Excel Test Case Workbook

Table 8: *Workbook structure*

Sheet	Content	Rows
Abdul-Rehman (24F-3056)	Test cases for models, stats, categories	62
Junaid Haider (24F-3122)	Test cases for csvutil, options, formatting	66
Summary & Traceability	Counts per module, data-driven totals, execution summary	—

6. Task 4: Suite Configuration, Ordering and Parallel Execution

6.1 Test Groups (Markers)

`pytest.ini` registers every marker and enables `--strict-markers`, so a mistyped marker fails immediately instead of silently selecting nothing.

Table 9: *Registered suites and their sizes*

Marker	Type	Focus	Tests
`models`	Module suite	Validation and dataclasses	20
`stats`	Module suite	Statistics and SVG geometry	30
`categories`	Module suite	Category colour mapping	12
`csvutil`	Module suite	CSV injection defence and encoding	22
`options`	Module suite	Export option parsing and filenames	17
`formatting`	Module suite	Currency, date and time formatting	27
`bva`	Technique suite	Boundary value analysis	18
`ecp`	Technique suite	Equivalence class partitioning	3
`security`	Technique suite	CSV injection tests (CWE-1236)	10
`data_driven`	Technique suite	Tests fed by external CSV files	16

6.2 Parameterization

Repetitive cases are expressed with `@pytest.mark.parametrize` and readable IDs that match the Excel test IDs (for example `AR-CAT-01` to `06`, `JH-FMT-01` to `06`, `CSV-01` to `08`, `JH-CSV-DD-01` to `08`). This keeps the tests data-driven and the reports traceable.

6.3 Execution Order

A `pytest_collection_modifyitems` hook in `sqe/tests/conftest.py` fixes the order of groups, and tests inside each module run in source order:

`models` → `stats` → `categories` → `csvutil` → `options` → `formatting`

6.4 Parallel Execution

`pytest-xdist` is used with `--dist loadgroup` so that all tests of one module run on the same worker (`xdist_group`). Parallel runs are started explicitly and are not forced in `pytest.ini`, because forcing them would break the deterministic order.

Ordering is guaranteed in serial runs only. On this small suite (128 fast tests) worker start-up costs more than it saves, so parallel runs are not faster than serial runs; the setup is provided to show the configuration and scales better with larger suites.

7. Execution and Verification Results

All commands were run from the project root. The results below were re-verified on the final project files.

Table 10: *Execution commands and verified results*

Purpose	Command	Result
Entire repository	`pytest`	241 passed
Assignment tests only	`pytest sqe`	128 passed
Parallel run	`pytest sqe -n auto --dist loadgroup`	128 passed
Abdul-Rehman only	`pytest sqe/tests/test_24F3056_*.py`	62 passed
Junaid Haider only	`pytest sqe/tests/test_24F3122_*.py`	66 passed
One suite	`pytest -m models` (similarly for each marker)	20 selected
Combined markers	`pytest -m "bva and csvutil"`	2 selected
Inspect order	`pytest sqe --collect-only -q`	128 collected, in the order in 6.3

Table 11: *Pass rate by group*

Group	Tests	Passed	Pass rate
Original regression suite	113	113	100%
New unit tests (Abdul-Rehman)	62	62	100%
New unit tests (Junaid Haider)	66	66	100%
Combined suite	**241**	**241**	**100%**

8. Deliverables and Traceability

Table 12: *Requirement-to-evidence mapping*

Task	Evidence in repository	Status
Task 1: Setup	`requirements-dev.txt`, `sqe/video/`	Complete
Task 2: Unit tests	`sqe/tests/test_24F3056_*.py`, `sqe/tests/test_24F3122_*.py`, `sqe/docs/design_notes_*.md`	Complete
Task 3: Excel + CSV	`sqe/docs/SQE_Assignment_3_Test_Cases.xlsx`, `sqe/tests/data/*.csv`	Complete

Task	Evidence in repository	Status
Summary report	sqe/docs/ f243122_f243056_SQE_Assignment3_FinalReport.pdf	Complete

9. Conclusion

The group delivered 128 new automated unit tests across six modules of the Expense Tracker, split evenly between two members, on top of the 113 original tests. Test design follows ECP, BVA and security techniques, every member contributes a CSV-driven test, and the suite can be run by module, by technique, by person, in a fixed order or in parallel. All 241 tests pass.